



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

SCHOOL OF TECHNOLOGY COLLEGE OF TECHNOLOGY

B.TECH COMPUTER ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

**Analysis and Design of Algorithms
(1010043316)**

Student Record Book

College Name :

Name of Student :

Roll No :

Semester :

Division :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

**COMPUTER ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**

VISION

To develop skilled professionals who can effectively tackle various challenges, equipped with advanced capabilities and ethical principles to innovate responsibly for the benefit of society.

MISSION

1. To provide top-notch, value-based technical education in all areas of computer engineering by utilizing cutting-edge tools and creative methods.
2. To create engineers with the necessary theoretical understanding and real-world experience who are moral, driven, and skillful.
3. To transmit the capacity to work both individually and together on simple to complicated challenges.



Course Outcomes (CO)

CO-1 : Analyze the asymptotic performance of algorithms.

CO-2 : Derive and solve recurrences describing the performance of divide-and-conquer algorithms.

CO-3: Find optimal solution by applying various methods.

CO-4 : Explain the major graph algorithms and their analyses. Employ graphs to model engineering problems, when appropriate.

CO-5 : Apply pattern matching algorithms to find particular patterns.

CO-6 : Differentiate polynomial and nonpolynomial problems.

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PS O1	PS O2	PS O3
CO1	2	2	2	2					1				1		
CO2	3	3	3	3					1			2	2		
CO3	3	3	3	3					1			2	2		
CO4	3	3	3	3					1			2		3	
CO5	2	3	2	3					1			2		3	
CO6	2	2	2	3					1					2	2



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

BONAFIDE CERTIFICATE

This is to certify that Mr./Ms
with Roll No.....from Semester..... Div..... has
successfully completed his/her laboratory experiments in the
ANALYSIS AND DESIGN OF ALGORITHMS (1010043316) from
the department of during the
academic year -

Staff In charge :

Head of Department:.....

Date of Examination:.....

Internal Examiner:

External Examiner :



TABLE OF CONTENT

Sr No.	Experiment Title	Page No		Date Of Start	Date of Completion	Sign	Marks (out of 10)
		From	To				
1	Implementation and Time analysis of sorting algorithms. a) Bubble sort b) Selection sort c) Insertion sort d) Merge sort e) Quick sort	01	16				
2	Implementation and Time analysis of linear and binary search algorithms.	17	24				
3	Implementation of max-heap sort algorithm.	25	28				
4	Implementation and Time analysis of factorial program using iterative and recursive method	29	31				
5	Implementation of a knapsack problem using dynamic programming.	32	34				
6	Implementation of chain matrix multiplication using dynamic programming.	35	38				
7	Implementation of making a change problem using dynamic programming.	39	41				
8	Implementation of a knapsack problem using greedy algorithms.	42	47				



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

9	Implementation of Graph and Searching (DFS and BFS)	48	52				
10	Implement prim's algorithm.	53	56				
11	Implement kruskal's algorithm.	57	59				
12	Implement the LCS problem.	60	62				



Date: _____

PRACTICAL - 1

AIM: Implementation and Time analysis of different sorting algorithms.

Bubble Sort:

Bubble Sort is a simple algorithm which is used to sort a given set of n elements provided in the form of an array with n number of elements. Bubble Sort compares all the element one by one and sorts them based on their values.

If the given array has to be sorted in ascending order, then bubble sort will start by comparing the first element of the array with the second element, if the first element is greater than the second element, it will swap both the elements, and then move on to compare the second and the third element, and so on. If we have total n elements, then we need to repeat this process for $n-1$ times.

It is known as bubble sort, because with every complete iteration the largest element in the given array, bubbles up towards the last place or the highest index, just like a water bubble rises up to the water surface.

Sorting takes place by stepping through all the elements one-by-one and comparing it with the adjacent element and swapping them if required.

Implementing Bubble Sort Algorithm

Following are the steps involved in bubble sort (for sorting a given array in ascending order):

- Starting with the first element (index = 0), compare the current element with the next element of the array.
- If the current element is greater than the next element of the array, swap them.
- If the current element is less than the next element, move to the next element. Repeat Step 1.

Complexity Analysis of Bubble Sort

Following are the Time and Space complexity for the Bubble Sort algorithm.

- Worst Case Time Complexity [Big-O]: $O(n^2)$
- Best Case Time Complexity [Big-omega]: $O(n)$
- Average Time Complexity [Big-theta]: $O(n^2)$
- Space Complexity: $O(1)$



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

AIM 1.1: Write a program to implement Bubble sort.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Selection Sort:

Selection sort is conceptually the simplest sorting algorithm. This algorithm will first find the smallest element in the array and swap it with the element in the first position, then it will find the second smallest element and swap it with the element in the second position, and it will keep on doing this until the entire array is sorted.

It is called selection sort because it repeatedly selects the next-smallest element and swaps it into the right place.

Implementing Selection Sort Algorithm

Following are the steps involved in selection sort (for sorting a given array in ascending order):

- Starting from the first element, we search the smallest element in the array, and replace it with the element in the first position.
- We then move on to the second position, and look for smallest element present in the subarray, starting from index 1, till the last index.
- We replace the element at the second position in the original array, or we can say at the first position in the subarray, with the second smallest element.
- This is repeated, until the array is completely sorted.

Complexity Analysis of Selection Sort

The following will be the time and space complexity for selection sort algorithm:

- Worst Case Time Complexity [Big-O]: $O(n^2)$
- Best Case Time Complexity [Big-omega]: $O(n^2)$
- Average Time Complexity [Big-theta]: $O(n^2)$
- Space Complexity: $O(1)$



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

AIM 1.2: Write a program to implement Selection sort.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output:



Insertion Sort:

The insertion sort works in a slightly different way. It always maintains a sorted sublist in the lower positions of the list. Each new item is then “inserted” back into the previous sublist such that the sorted sublist is one item larger. The shaded items represent the ordered sublists as the algorithm makes each pass.

Implementing Selection Sort Algorithm

Following are the steps involved in insertion sort:

- We start by making the second element of the given array, i.e. element at index 1, the key. The key element here is the new number that we need to add to our existing sorted set of numbers
- We compare the key element with the element(s) before it, in this case, element at index 0:
- If the key element is less than the first element, we insert the key element before the first element.
- If the key element is greater than the first element, then we insert it after the first element.
- Then, we make the third element of the array as key and will compare it with elements to its left and insert it at the right position.
- And we go on repeating this, until the array is sorted.

Complexity Analysis of Selection Sort

- Worst Case Time Complexity [Big-O]: $O(n^2)$
- Best Case Time Complexity [Big-omega]: $O(n)$
- Average Time Complexity [Big-theta]: $O(n^2)$
- Space Complexity: $O(1)$



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

AIM 1.3: Write a program to implement Insertion sort.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Merge Sort:

Merge sort is a divide-and-conquer algorithm based on the idea of breaking down a list into several sub-lists until each sub-list consists of a single element and merging those sub-lists in a manner that results in a sorted list.

Implementing Merge Sort Algorithm:

- Divide the unsorted list into N sublists, each containing 1 element.
- Take adjacent pairs of two singleton lists and merge them to form a list of 2 elements. N will now convert into $N/2$ lists of size 2.
- Repeat the process till a single sorted list is obtained.

While comparing two sublists for merging, the first element of both lists is taken into consideration. While sorting in ascending order, the element that is of a lesser value becomes a new element of the sorted list. This procedure is repeated until both the smaller sublists are empty and the new combined sublist comprises all the elements of both the sublists.

Algorithm:

MergeSort(arr[], l, r) If $r > l$

Find the middle point to divide the array into two halves: middle $m = (l+r)/2$

Call mergeSort for first half: Call mergeSort(arr, l, m)

Call mergeSort for second half: Call mergeSort(arr, m+1, r)

Merge the two halves sorted in step 2 and 3: Call merge(arr, l, m, r)

Complexity Analysis of Merge Sort

The following will be the time and space complexity for merge sort algorithm:

- Worst Case Time Complexity [Big-O]: $O(n \log n)$
- Best Case Time Complexity [Big-omega]: $O(n \log n)$
- Average Time Complexity [Big-theta]: $O(n \log n)$
- Space Complexity: $O(n)$



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

AIM 1.4 : Write a program to implement Merge sort.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output:



Quick Sort:

Quick sort is based on the divide-and-conquer approach based on the idea of choosing one element as a pivot element and partitioning the array around it such that: Left side of pivot contains all the elements that are less than the pivot element Right side contains all elements greater than the pivot.

It reduces the space complexity and removes the use of the auxiliary array that is used in merge sort. Selecting a random pivot in an array results in an improved time complexity in most of the cases.

Implementing Quick Sort Algorithm:

- Select the first element of array as the pivot element First, we will see how the partition of the array takes place around the pivot.
- Choose the highest index value has pivot
- Take two variables to point left and right of the list excluding pivot
- left points to the low index
- right points to the high
- while value at left is less than pivot move right
- while value at right is greater than pivot move left
- if both step 5 and step 6 does not match swap left and right
- if $\text{left} \geq \text{right}$, the point where they met is new pivot

Algorithm:

```
quickSort(left, right) if right-left <= 0
return else
pivot = A[right]
partition = partitionFunc(left, right, pivot) quickSort(left,partition-1)
quickSort(partition+1,right)
end if
end procedure
```

Complexity Analysis of Merge Sort:

The following will be the time and space complexity for merge sort algorithm:

- Worst Case Time Complexity [Big-O]: $O(n^2)$
- Best Case Time Complexity [Big-omega]: $O(n \log n)$
- Average Time Complexity [Big-theta]: $O(n \log n)$
- Space Complexity: $O(n)$



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

AIM 1.5: Write a program to implement Quick sort.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Solve the below mention questionnaires:

1. The way a card game player arranges his cards as he picks them one by one can be compared to
 - a) Quick sort
 - b) Merge sort
 - c) Insertion sort
 - d) Bubble sort
2. The correct order of the efficiency of the following sorting algorithms according to their overall running time comparison is
 - a) Insertion>selection>bubble
 - b) Insertion>bubble>selection
 - c) Selection>bubble>insertion.
 - d) bubble>selection>insertion
3. For the improvement of efficiency of quick sort the pivot can be
 - a) the first element
 - b) the mean element
 - c) the last element
 - d) None of the above
4. Which of the following is an example of in-place algorithm?
 - a) Merge Sort
 - b) Bubble Sort
 - c) Heap Sort
 - d) Quick Sort
5. Quick Sort can be categorized into which of the following?
 - a) Brute Force technique
 - b) Dynamic programming
 - c) Greedy algorithm
 - d) Divide and conquer

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL - 2

AIM: Implementation and Time analysis of linear and binary search algorithms.

Linear Search Algorithm:

Linear search algorithm finds a given element in a list of elements with $O(n)$ time complexity where n is the total number of elements in the list. This search process starts comparing the search element with the first element in the list. If both are matching then results with the element found, otherwise the search element is compared with the next element in the list. If both are matched, then the result is "element found". Otherwise, repeat the same with the next element in the list until the search element is compared with the last element in the list, if that last element also doesn't match, then the result is "Element not found in the list". That means, the search element is compared with element by element in the list.

Implementing Linear Search Algorithm:

- Read the search element from the user
- Compare the search element with the first element in the list.
- If both are matching, then display "Given element found!!!" and terminate the function
- If both are not matching, then compare the search element with the next element in the list.
- Repeat steps 3 and 4 until the search element is compared with the last element in the list.
- If the last element in the list also doesn't match, then display "Element not found!!!" and terminate the function.

Algorithm:

```
procedure linear_search (list, value) for each item in the list
if match item == value return the item's location
end if end for
end procedure
```

Complexity Analysis of Linear Search:

- Worst Case Time Complexity [Big-O]: $O(n)$
- Best Case Time Complexity [Big-omega]: $O(1)$
- Average Time Complexity [Big-theta]: $O(n)$
- Space Complexity: $O(1)$



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

AIM 2.1 : Write a program to implement Linear Search. Code:

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Binary Search Algorithm:

The binary search algorithm can be used with only a sorted list of elements. That means, binary search can be used only with a list of elements which are already arranged in an order. The binary search cannot be used for lists of elements which are in random order. This search process starts comparing the search element with the middle element in the list. If both are matched, then the result is "element found". Otherwise, we check whether the search element is smaller or larger than the middle element in the list. If the search element is smaller, then we repeat the same process for the left sublist of the middle element. If the search element is larger, then we repeat the same process for the right sublist of the middle element. We repeat this process until we find the search element in the list or until we left with a sublist of only one element. And if that element also doesn't match with the search element, then the result is "Element not found in the list".

Implementing Binary Search Algorithm:

- Read the search element from the user
- Find the middle element in the sorted list
- Compare the search element with the middle element in the sorted list.
- If both are matching, then display "Given element found!!!" and terminate the function
- If both are not matching, then check whether the search element is smaller or larger than the middle element.
- If the search element is smaller than the middle element, then repeat steps 2, 3, 4 and 5 for the left sublist of the middle element.
- If the search element is larger than the middle element, then repeat steps 2, 3, 4 and 5 for the right sublist of the middle element.
- Repeat the same process until we find the search element in the list or until the sublist contains only one element.
- If that element also doesn't match with the search element, then display "Element not found in the list!!!" and terminate the function.

Algorithm:

```
Procedure binary_search A ← sorted array
n ← size of array
x ← value to be searched
Set lowerBound = 1 Set upperBound = n
while x not found
if upperBound < lowerBound EXIT: x does not exists.
set midPoint = (lowerBound + upperBound) / 2
if A[midPoint] < x
set lowerBound = midPoint + 1
if A[midPoint] > x
set upperBound = midPoint - 1
```



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

if $A[\text{midPoint}] == x$
EXIT: x found at location midPoint end while
end procedure

Complexity Analysis of Binary Search:

- Worst Case Time Complexity [Big-O]: $O(\log n)$
- Best Case Time Complexity [Big-omega]: $O(1)$
- Average Time Complexity [Big-theta]: $O(\log n)$
- Space Complexity: $O(1)$



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

AIM 2.2 : Write a program to implement Binary Search Algorithm.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :

PRACTICAL- 3

AIM: Implementation of max-heap sort algorithm.

Explanation: Heaps can be used in sorting an array. In max-heaps, maximum element will always be at the root. Heap Sort uses this property of heap to sort the array. It is similar to selection sort where we first find the maximum element and place the maximum element at the end. We repeat the same process for remaining elements.

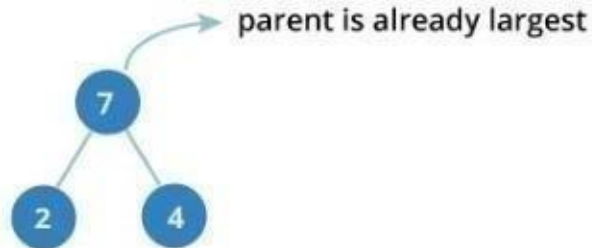
Heap Sort Algorithm for sorting in increasing order:

- Build a max heap from the input data.
- At this point, the largest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of heap by 1. Finally, heapify the root of tree.
- Repeat above steps while size of heap is greater than 1.

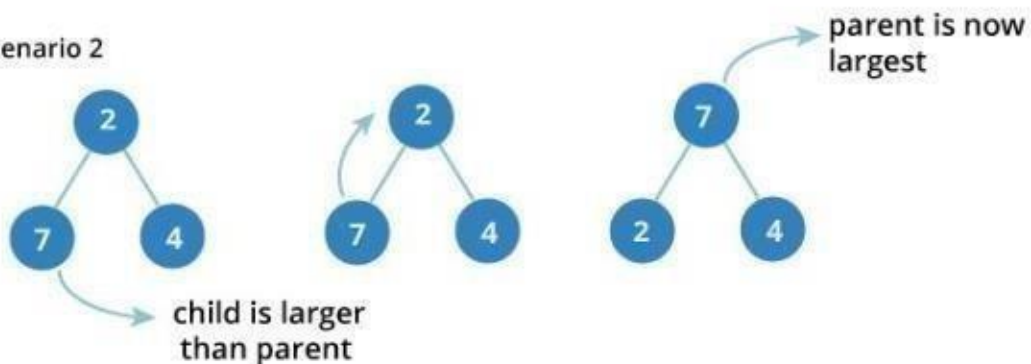
How to build the heap?

Heapify procedure can be applied to a node only if its children nodes are heapified. So the heapification must be performed in the bottom up order.

Scenario 1



Scenario 2





**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Code :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Solve the below mention questionnaires:

- 1) In a max-heap, the element with the greatest key is always in which node?
a) Leaf node
b) First node of left subtree
c) Root node
d) First node of right subtree
- 2) Heap can be used as...
a) Priority queue
b) Stack
c) A decreasing order array
d) None of the mentioned
- 3) An array consists of n elements. We want to create a heap using the elements. The time complexity of building a heap will be in order of
a) $O(n*n*\log n)$
b) $O(n*\log n)$
c) $O(n*n)$
d) $O(n * \log n * \log n)$
- 4) The max heap constructed from the list of numbers 30, 10, 80, 60, 15, 55 is
a) 80, 55, 60, 15, 10, 30
b) 60, 80, 55, 30, 10, 15
c) 80, 60, 55, 30, 10, 15
d) None
- 5) Always heap is a
a) Binary search tree
b) Full binary tree
c) None
d) Complete binary tree

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL - 4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method

Explanation: Factorial of a non-negative integer, is multiplication of all positive integers smaller than or equal to n. For example factorial of 6 is $6*5*4*3*2*1$ which is 720. A factorial is represented by a number and a "!" mark at the end. It is widely used in permutations and combinations to calculate the total possible outcomes. A French mathematician Christian Kramp firstly used the exclamation.

Iterative Solution

Factorial can also be calculated iteratively as recursion can be costly for large numbers. Here we have shown the iterative approach using both for and while loop.

Recursive Solution

Factorial can be calculated using following recursive formula.

$$n! = n * (n-1)!$$

$$n! = 1 \text{ if } n = 0 \text{ or } n = 1$$

Code :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Solve the below mention questionnaires:

1) Which of the following statements is true regarding the iterative implementation of a factorial program?

- a) The iterative method uses more memory compared to the recursive method.
- b) The iterative method has a time complexity of $O(n!)$.
- c) The iterative method avoids the overhead of multiple function calls.
- d) The iterative method is slower than the recursive method.

2) What is the time complexity of the recursive method for calculating the factorial of a number?

- a) $O(1)$
- b) $O(n)$
- c) $O(n \log n)$
- d) $O(n!)$

3) In the context of factorial computation, which of the following is a disadvantage of using the recursive method over the iterative method?

- a) The recursive method is harder to implement.
- b) The recursive method may lead to stack overflow for large values of n .
- c) The recursive method has a better space complexity.
- d) The recursive method is inherently faster than the iterative method.

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL– 5

AIM : Implementation of a knapsack problem using dynamic programming.

Explanation : The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a weight and a profit value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total profit value is as large as possible. This is a 0-1 knapsack problem hence we can either take an entire item or reject it completely. We cannot break an item and fill the knapsack.

- In this problem we have a Knapsack that has a weight limit W , which represents knapsack capacity.
- Given two integer arrays $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items respectively.
- Find out the maximum value subset of $val[]$ such that the sum of the weights of this subset is smaller than or equal to W .

Code :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Solve the below mention questionnaires:

1) You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights {20, 30, 40, 70} and values {70, 80, 90, 200}. What is the maximum value of the items you can carry using the knapsack?

- a) 160
- b) 200
- c) 170
- d) 90

2) What is the time complexity of the above dynamic programming implementation of the Knapsack problem with n items and a maximum weight of W ?

- a) $O(n)$
- b) $O(n + w)$
- c) $O(nW)$
- d) $O(n^2)$

3) The 0-1 Knapsack problem can be solved using the Greedy algorithm.

- a) True
- b) False

4) 0/1 knapsack is based on

- a) Greedy method
- b) Branch and bound method
- c) Dynamic programming
- d) Divide and conquer

5) Bin-packing problem is the application of

- a) Knapsack
- b) Branch and bound
- c) Back tracking
- d) Dynamic programming

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL – 6

AIM : Implementation of chain matrix multiplication using dynamic programming.

Explanation: Given a sequence of matrices, find the most efficient way to multiply these matrices together. The problem is not actually to perform the multiplications, but merely to decide in which order to perform the multiplications.

We have many options to multiply a chain of matrices because matrix multiplication is associative. In other words, no matter how we parenthesize the product, the result will be the same. For example, if we had four matrices A, B, C, and D, we would have:

$$(ABC) D = (AB) (CD) = A (BCD) = \dots$$

However, the order in which we parenthesize the product affects the number of simple arithmetic operations needed to compute the product, or the efficiency. For example, suppose A is a 10×30 matrix, B is a 30×5 matrix, and C is a 5×60 matrix. Then,

$$\begin{aligned} (AB) C &= (10 \times 30 \times 5) + (10 \times 5 \times 60) = 1500 + 3000 = 4500 \text{ operations} \\ A (BC) &= (30 \times 5 \times 60) + (10 \times 30 \times 60) = 9000 + 18000 = 27000 \text{ operations.} \end{aligned}$$

Clearly the first parenthesizing requires less number of operations.

Given an array $p[]$ which represents the chain of matrices such that the i th matrix A_i is of dimension $p[i-1] \times p[i]$. We need to write a function `MatrixChainOrder()` that should return the minimum number of multiplications needed to multiply the chain.

This problem can be solved using Dynamic Programming. First we will compute results for sub- chains of the original chain and store these results in a 2-D array. Then we will use these results to compute results for larger chains.



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Code :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Solve the below mention questionnaires :

1) Consider the two matrices P and Q which are 10×20 and 20×30 matrices respectively. What is the number of multiplications required to multiply the two matrices?

- a) $10 * 20$
- b) $10 * 30$
- c) $20 * 30$
- d) $10 * 20 * 30$

2) What is the time complexity of the above dynamic programming implementation of the matrix chain problem?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n^3)$

3) Which of the following methods can be used to solve the matrix chain multiplication problem?

- Dynamic programming
- Brute force
- c) Recursion
- d) All of the mentioned

4) Consider the matrices P, Q and R which are 10×20 , 20×30 and 30×40 matrices respectively. What is the minimum number of multiplications required to multiply the three matrices?

- a) 18000
- b) 12000
- c) 24000
- d) 32000

5) Consider the matrices P, Q, R and S which are 20×15 , 15×30 , 30×5 and 5×40 matrices respectively. What is the minimum number of multiplications required to multiply the four matrices?

- a) 6050
- b) 7500
- c) 7750
- d) 12000

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL -7

AIM: Implementation of making a change problem using dynamic programming.

Explanation:

Given a value N , if we want to make change for N cents, and we have infinite supply of each of

$S = \{ S_1, S_2, \dots, S_m \}$ valued coins, how many ways can we make the change? The order of coins doesn't matter.

For example, for $N = 4$ and $S = \{1, 2, 3\}$, there are four solutions: $\{1, 1, 1, 1\}, \{1, 1, 2\}, \{2, 2\}, \{1, 3\}$. So output should be 4. For $N = 10$ and $S = \{2, 5, 3, 6\}$, there are five solutions: $\{2, 2, 2, 2, 2\}, \{2, 2, 3, 3\}, \{2, 2, 6\}, \{2, 3, 5\}$ and $\{5, 5\}$. So the output should be 5.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Give the Answer to the below Short Questions:

1) What is the time complexity of making changes using dynamic programming?

2) If $S = 5$ and $N = \{1, 2, 3\}$, means we have a sum of 5 and want to make change for this sum using coins of denomination 1, 2 and 3. There is an infinite supply of these coins. We have to find out how many ways we can make this change.

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL- 8

AIM: Implementation of a knapsack problem using greedy algorithms.

Description:

Given a set of items, each with a weight and a value, determine a subset of items to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

The knapsack problem is in combinatorial optimization problem. It appears as a subproblem in many, more complex mathematical models of real-world problems. One general approach to difficult problems is to identify the most restrictive constraint, ignore the others, solve a knapsack problem, and somehow adjust the solution to satisfy the ignored constraints.

Problem Scenario

A thief is robbing a store and can carry a maximal weight of W into his knapsack. There are n items available in the store and weight of i th item is w_i and its profit is p_i . What items should the thief take?

In this context, the items should be selected in such a way that the thief will carry those items for which he will gain maximum profit. Hence, the objective of the thief is to maximize the profit.

Based on the nature of the items, Knapsack problems are categorized as

- Fractional Knapsack
- Knapsack

Fractional Knapsack

In this case, items can be broken into smaller pieces, hence the thief can select fractions of items. According to the problem statement,

- There are n items in the store
- Weight of i th item $w_i > 0$
- Profit for i th item $p_i > 0$ and
- Capacity of the Knapsack is W

In this version of Knapsack problem, items can be broken into smaller pieces. So, the thief may take only a fraction x_i of i th item.

$$0 \leq x_i \leq 1$$

The i th item contributes the weight $x_i \cdot w_i$ to the total weight in the knapsack and profit $x_i \cdot p_i$ to the total profit.

Hence, the objective of this algorithm is to

$$\text{maximize } \sum_{i=1}^n x_i \cdot p_i$$

$$\text{maximize } \sum_{i=1}^n x_i \cdot p_i$$



subject to constraint,

$$\sum_{i=1}^n x_i w_i \leq W$$

It is clear that an optimal solution must fill the knapsack exactly, otherwise we could add a fraction of one of the remaining items and increase the overall profit.

Thus, an optimal solution can be obtained by

$$\sum_{i=1}^n x_i w_i = W$$

In this context, first we need to sort those items according to the value of p_i/w_i , so that $p_i/w_i \geq p_{i+1}/w_{i+1}$. Here, x is an array to store the fraction of items.

Algorithm: Greedy-Fractional-Knapsack ($w[1..n]$, $p[1..n]$, W)
for $i = 1$ to n
do $x[i] = 0$ weight = 0
if weight + $w[i] \leq W$ then $x[i] = 1$
weight = weight + $w[i]$ else
 $x[i] = (W - \text{weight}) / w[i]$ weight = W
break return x

Analysis

If the provided items are already sorted into a decreasing order of p_i/w_i , then the while loop takes a time in $O(n)$; Therefore, the total time including the sort is in $O(n \log n)$.



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Code :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output:



Give the Answer to the below Short Questions:

1) The Knapsack problem is an example of

- a) Greedy algorithm
- b) Divide and conquer
- c) 2D dynamic programming
- d) 1D dynamic programming

2) Which of the following methods can be used to solve the Knapsack problem?

- a) Brute force algorithm
- b) Dynamic programming
- c) Recursion
- d) Brute force, Recursion and Dynamic Programming

3) You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights {20, 30, 40, 70} and values {70, 80, 90, 200}. What is the maximum value of the items you can carry using the knapsack?

- a) 160
- b) 170
- c) 200
- d) 90

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL-9

AIM: Implementation of Graph and Searching (DFS and BFS)

Explanation:

A standard DFS implementation puts each vertex of the graph into one of two categories:

- Visited
- Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The DFS algorithm works as follows:

- Start by putting any one of the graph's vertices on top of a stack.
- Take the top item of the stack and add it to the visited list.
- Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of stack.
- Keep repeating steps 2 and 3 until the stack is empty.

Aim 9.1: Write a program to implement DFS.

Code :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Traversal means visiting all the nodes of a graph. Breadth first traversal or Breadth first Search is a recursive algorithm for searching all the vertices of a graph or tree data structure. In this article, you will learn with the help of examples the BFS algorithm, BFS pseudocode and the code of the breadth first search algorithm with implementation in C++, C, Java and Python programs.

BFS algorithm

A standard DFS implementation puts each vertex of the graph into one of two categories:

- Visited
- Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles. The algorithm works as follows:

- Start by putting any one of the graph's vertices at the back of a queue.
- Take the front item of the queue and add it to the visited list.
- Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
- Keep repeating steps 2 and 3 until the queue is empty.

The graph might have two different disconnected parts so to make sure that we cover every vertex, we can also run the BFS algorithm on every node.

Aim 9.2: Write a program to implement BFS

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Give the Answer to Questions:

- 1) Depth First Search is equivalent to which of the traversals in the Binary Trees?
 - a) Pre-order Traversal
 - b) Post-order Traversal
 - c) Level-order Traversal
 - d) In-order Traversal
- 2) Time Complexity of DFS is? (V – number of vertices, E – number of edges)
- 3) The Depth First Search traversal of a graph will result in?
- 4) Breadth First Search is equivalent to which of the traversals in the Binary Trees?

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL -10

AIM: Implement prim's algorithm.

Explanation:

The idea behind Prim's algorithm is simple, a spanning tree means all vertices must be connected. So the two disjoint subsets (discussed above) of vertices must be connected to make a Spanning Tree. And they must be connected with the minimum weight edge to make it a Minimum Spanning Tree.

Algorithm

- Create a set mstSet that keeps track of vertices already included in MST.
- Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign key value as 0 for the first vertex so that it is picked first.
- While mstSet doesn't include all vertices
- Pick a vertex u which is not there in mstSet and has minimum key value.
- Include u to mstSet.
- Update key value of all adjacent vertices of u. To update the key values, iterate through all adjacent vertices. For every adjacent vertex v, if weight of edge u-v is less than the previous key value of v, update the key value as weight of u-v
- The idea of using key values is to pick the minimum weight edge from cut. The key values are used only for vertices which are not yet included in MST, the key value for these vertices indicate the minimum weight edges connecting them to the set of vertices included in MST.



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Give the Answer to the below Questions:

1). What algorithm technique is used in the implementation of Prim's solution for the MST?

- a) Greedy Technique
- b) Divide-and-Conquer Technique
- c) Dynamic Programming Technique
- d) The algorithm combines more than one of the above techniques

2) The output of Prim's algorithm is.....

3) For minimum Spanning Tree construction, Prim's algorithm selects an edge..

- a) That adds a new vertex to partially constructed tree with minimal increment in cost of MST
- b) With maximum number of vertices connected to it so that MST has least diameter
- c) With minimum weight so that the cost of MST is always minimum.
- d) That does not introduce a cycle.

Conclusion :

Signature with Date of Completion	
Marks out of 10	



Date: _____

PRACTICAL -11

Aim: Implement Kruskal's algorithm.

Explanation:

Below are the steps for finding MST using Kruskal's algorithm

- Sort all the edges in non-decreasing order of their weight.
- Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If cycle is not formed, include this edge. Else, discard it.
- Repeat step#2 until there are $(V-1)$ edges in the spanning tree.

Code :



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Give the Answer to the below Questions:

1)What is the time complexity to extract a vertex from the priority queue in Krushkal's algorithm?
a) $\log(V)$
b) $V.V$
c) $E.E$
d) $E \log(E)$

2)The output of Krushkal's algorithm is

3)Assume a graph has 10 vertices and 20 edges. In Krushkal's minimum spanning tree method, 5 edges are rejected. How many edges are not considered during execution of the algorithm on the given graph?

- a) 6
- b) 5
- c) 4
- d) 10

Conclusion :

Signature with Date of Completion	
Marks out of 10	



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Date: _____

PRACTICAL-12

Aim: Implement LCS problem.

Explanation: Given two sequences, find the length of the longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily contiguous. For example, “abc”, “abg”, “bdf”, “aeg”, “acefg”, .. etc are subsequences of “abcdefg”.

Code:



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION

Output :



Give the Answer to the below Questions:

1) What is the time complexity of the dynamic programming approach for finding the Longest Common Subsequence (LCS) of two strings of length m and n ?

- a) $O(m + n)$
- b) $O(m * n)$
- c) $O(m^2 * n^2)$
- d) $O(2^n)$

2) Which of the following is true about the Longest Common Subsequence (LCS) problem?

- a) The LCS of two sequences is always unique.
- b) LCS can be found using a greedy algorithm.
- c) LCS can be used to measure the similarity between two sequences.
- d) LCS always includes all characters of the shorter sequence.

3) Given two sequences "ABCD" and "ACBAD", what is the length of their Longest Common Subsequence (LCS)?

- a) 2
- b) 3
- c) 4
- d) 5

Conclusion :

Signature with Date of Completion	
Marks out of 10	